Departamento de Informática, Univ. de Valencia

Sistemas Inteligentes

Juan Domingo
Juan.Domingo@uv.es

Septiembre de 2026

Tema 4:

Representación del conocimiento

Índice del tema 4: comienzo de la sección 1

1 Representación y razonamiento en AI

2 Representación y razonamiento con conocimiento preciso
2.1 Cálculo proposicional
2.2 Cálculo de predicados
2.3 Hechos y reglas
2.4 Sistemas de producción

3 Representación y razonamiento con conocimiento impreciso o incompleto
3.1 Necesidad del conocimiento impreciso
3.2 Modelos empíricos
3.3 Modelos probabilistas
3.4 Modelos posibilistas

Introducción (I)

Los programas de ordenador que resuelven los problemas abordables por métodos convencionales se basan en

Algoritmos + Estructuras de datos = Programas
(que encuentran la solución)

Sin embargo, hemos visto que para los problemas típicos en AI los algoritmos "clásicos" (aquellos que usan sólo la naturaleza y estructura de los datos de entrada) no son suficientes. Se requiere poder codificar (programar) reglas acerca del ambiente, las restricciones, etc..; es decir, se requiere **conocimiento (heurístico)**.
Pero la proliferación de reglas y restricciones trae una explosión combinatoria, y se requiere alguna manera de poder aplicar esas reglas de modo eficiente. Esto es el **razonamiento**. Por eso, en AI tenemos que

Conocimiento + Razonamiento + Datos = Programas de AI
(que encuentran, o tratan de encontrar, una solución)

Introducción (II)

En AI, es necesario encontrar una forma de separar la representación del conocimiento de los algoritmos que lo manejan.

Esto requiere estructuras de datos que puedan acomodar conocimiento genérico (de campos del saber o de problemas distintos) y algoritmos que puedan actuar sobre esas estructuras, sin que importe su contenido específico (es decir, si lo que se modeliza con ellas es un juego, un tratamiento médico, una inversión en bolsa, ...). Estos algoritmos son el **razonamiento** y su función es extraer conclusiones y/o realizar predicciones.

Veremos primero las formas de representación del conocimiento preciso (el que se conoce sin ninguna ambigüedad) y de razonamiento con él y después las de tratan de abordar el problema del conocimiento impreciso o incompleto y de la propagación de esta imprecisión a las conclusiones y predicciones generadas con él.

Índice del tema 4: comienzo de la sección 2

1 Representación y razonamiento en AI

2 Representación y razonamiento con conocimiento preciso
2.1 Cálculo proposicional
2.2 Cálculo de predicados
2.3 Hechos y reglas
2.4 Sistemas de producción

3 Representación y razonamiento con conocimiento impreciso o incompleto
3.1 Necesidad del conocimiento impreciso
3.2 Modelos empíricos
3.3 Modelos probabilistas
3.4 Modelos posibilistas

Cálculo lógico: definiciones

Proposición atómica: una afirmación o negación que no puede ser descompuesta. Trata de representar enunciados lingüísticos que se suponen sin estructura interna. Ej.: "Me gusta la lógica", "Algunas clases son aburridísimas", "Todo hombre es mortal", etc.
Fórmula válida: una cadena de símbolos conteniendo sólo proposiciones atómicas y símbolos de una lista finita conocida que sea sintácticamente correcta en alguna sintáxis bien definida.
Axioma: una fórmula válida que pertenece a un conjunto inicial dado.
Regla de deducción: regla sintáctica para la generación de una fórmula válida llamada **consecuencia lógica** a partir de otras fórmulas válidas llamadas **premisas**.
Teorema: toda consecuencia lógica cuyas premisas son axiomas, otros teoremas o ambos.
Cálculo: un conjunto finito de reglas sintácticas para definir algorítmicamente qué es una fórmula válida, más un conjunto finito de fórmulas declaradas como axiomas más un conjunto finito de reglas de deducción.

Cálculo proposicional (CProp) (I)

Proposición (del CProp): una afirmación o negación que involucra a una única entidad expresamente mencionada. Ej.: "Me gusta la lógica", "Hoy no ha llovido",.. pero no "Todos los pájaros vuelan" o "Algunas aves nadan". Cada proposición se representará por una letra (letra proposicional): a, b, c... Si las agotásemos, emplearemos símbolos únicos como a1, a2,...que llamaremos también 'letras'.

Símbolos válidos: los símbolos válidos del CProp son las letras proposicionales y las conectivas lógicas & (and), $\vee$ (or), $\neg$ (not) e $\supset$ (implica).

(Los paréntesis, que se usarán por claridad, no son estrictamente necesarios si definimos prioridades para las conectivas)

Fórmula válida:

- Una letra proposicional es una fórmula válida.
- Si A y B son fórmulas
  1. $A\&B$ es una fórmula
  2. $A \vee B$ es una fórmula
  3. $\neg A$ es una fórmula
  4. $A \supset B$ es una fórmula
- No existen más fórmulas proposicionales que las ya definidas.

Cálculo proposicional (CProp) (II)

Los **axiomas** del cálculo proposicional son:
Si A, B y C son fórmulas proposicionales cualesquiera,

1. $A \supset (B \supset A)$
2. $(A \supset B) \supset ((A \supset (B \supset C)) \supset (A \supset C))$
3. $A \supset (B \supset A\&B)$
4. $A\&B \supset A$
5. $A\&B \supset B$
6. $A \supset A \vee B$
7. $B \supset A \vee B$
8. $(A \supset C) \supset ((B \supset C) \supset (A \vee B \supset C))$
9. $(A \supset B) \supset ((A \supset \neg B) \supset \neg A)$
10. $\neg\neg A \supset A$

Cálculo proposicional (CProp) (III)

La única **regla de deducción** del cálculo proposicional es

$$\frac{A, A \supset B}{B}$$

siendo A y B cualesquiera fórmulas válidas, que se lee: "B es consecuencia inmediata de A y de $A \supset B$".

Se dice que una fórmula D del cálculo es **deducible** de un conjunto finito de fórmulas, $\{A_1, \ldots, A_n\}$, lo que se denota como $A_1, \ldots, A_n \vdash D$ si:

- D es una de las fórmulas A_i
  o bien
- D es consecuencia inmediata de E y F siendo E y F deducibles de $\{A_1, \ldots, A_n\}$

No existen más fórmulas deducibles del conjunto $\{A_1, \ldots, A_n\}$ que las obtenidas según se ha indicado.

Cálculo proposicional (CProp) (IV)

Una fórmula F es un **teorema formal** (se dice también: es **formalmente demostrable**, o simplemente es demostrable) en un cálculo si es deducible de los axiomas del cálculo. En ese caso se escribe $\vdash F$

Una **demostración** de D es una cadena finita de fórmulas tales que

- Su último elemento es D
- Cada elemento es o bien un axioma, o es consecuencia inmediata de otros elementos anteriores de la cadena

Se dice que un cálculo es **consistente** si para ninguna fórmula F las fórmulas F y $\neg F$ son ambas demostrables.

Se dice que un cálculo es **completo** respecto a cierta propiedad $\mathcal{P}$ de las fórmulas si para toda fórmula F que posea la propiedad $\mathcal{P}$ se cumple que F es demostrable.

Cálculo proposicional (CProp) (IV)

Ejemplo de demostración:
Sea D cualquier fórmula válida. La siguiente secuencia de cinco fórmulas es una demostración de $D \supset D$:

F1 $D \supset (D \supset D)$
(Axioma 1 con $A \equiv D$ y $B \equiv D$)

F2 $(D \supset (D \supset D)) \supset ([D \supset ((D \supset D) \supset D)] \supset [D \supset D])$
(Axioma 2 con $A \equiv D$, $B \equiv D \supset D$ y $C \equiv D$)

F3 $[D \supset ((D \supset D) \supset D)] \supset [D \supset D]$
(Regla de deducción aplicada a F1 y F2)

F4 $D \supset ((D \supset D) \supset D)$
(Axioma 1 con $A \equiv D$ y $B \equiv D \supset D$)

F5 $D \supset D$
(Regla de deducción aplicada a F4 y F3)

Como se puede ver, este medio de demostración no es aplicable en la práctica para fórmulas mínimamente complejas.

Cálculo proposicional (CProp) (V)

Interpetación del CProp:
A cada letra proposicional se le asignará uno de los dos valores, $\mathcal{T}$ ó $\mathcal{F}$, cada vez.
A cada fórmula compuesta se le asignará uno de los dos valores, de acuerdo a las reglas de las siguientes tablas:

A	$\neg A$
$\mathcal{T}$	$\mathcal{F}$
$\mathcal{F}$	$\mathcal{T}$

A	B	$A\&B$
$\mathcal{T}$	$\mathcal{T}$	$\mathcal{T}$
$\mathcal{T}$	$\mathcal{F}$	$\mathcal{F}$
$\mathcal{F}$	$\mathcal{T}$	$\mathcal{F}$
$\mathcal{F}$	$\mathcal{F}$	$\mathcal{F}$

A	B	$A \vee B$
$\mathcal{T}$	$\mathcal{T}$	$\mathcal{T}$
$\mathcal{T}$	$\mathcal{F}$	$\mathcal{T}$
$\mathcal{F}$	$\mathcal{T}$	$\mathcal{T}$
$\mathcal{F}$	$\mathcal{F}$	$\mathcal{F}$

A	B	$A \supset B$
$\mathcal{T}$	$\mathcal{T}$	$\mathcal{T}$
$\mathcal{T}$	$\mathcal{F}$	$\mathcal{F}$
$\mathcal{F}$	$\mathcal{T}$	$\mathcal{T}$
$\mathcal{F}$	$\mathcal{F}$	$\mathcal{T}$

Se dice que una fórmula F del cálculo en que intervienen las letras proposicionales $\{a_1, \ldots, a_n\}$ es una **tautología** si se le asigna el valor $\mathcal{T}$ para cualquiera de las 2^n posibles asignaciones de $\{\mathcal{T}, \mathcal{F}\}$ a cada una de las a_i.

Observación importante: la tabla de verdad de $A \supset B$ es la misma que la de $\neg A \vee B$.

Cálculo proposicional (CProp) (VI)

Se cumple que

- Todos los axiomas son tautologías.
- Si A y $A \supset B$ son tautologías, B lo es.

lo cual se puede comprobar directamente constuyendo las tablas de verdad para cada axioma, y viendo la tabla de $A \supset B$.
De estas dos propiedades se deduce que todo teorema es una tautología (porque la propiedad "ser una tautología" se propaga al aplicar la regla de deducción). Por medios más complicados se puede también probar que toda tautología es un teorema.

De esto se deduce que el cálculo proposicional es consistente, y también completo con respecto a la propiedad $\mathcal{P}(F) \equiv \{F \textit{ es una tautología}\}$.

Acudiendo ahora a una interpretación más familiar: toda proposición del cálculo proposicional es demostrable si y sólo si es siempre verdadera, independientemente de los valores de verdad de sus proposiciones atómicas constituyentes.

Cálculo de predicados (CPre) (I)

Trata de representar enunciados lingüísticos (proposiciones) que expresan propiedades de varios indivíduos, y que se cumplen o no según el valor particular que tome cada indivíduo.

Ej.: "X es un alumno de doctorado", "Todo hombre es mortal", "X es hermano de Y", "X,Y y Z se conocen entre sí",...

Trataremos sólo el CPre de **primer** orden, en la cual los objetos y las proposiciones pertenecen a conjuntos diferentes, y las variables de las proposiciones toman valores en el conjunto de objetos.

Esto elimina problemas de autoreferencialidad, como "Epiménides dice que los cretenses mienten" (Epiménides es cretense), o "Esta proposición es falsa".

Cálculo de predicados (CPre) (II)

Se supondrá que aceptamos alguna convención léxica por la cual podemos diferenciar las variables de lo que no lo son. Sean los símbolos que comienzan por x, y o z como $\{x, y, z, \ldots, x1, x2, \ldots, y1, \ldots\}$ variables. Entonces:

- Una letra proposicional es una fórmula válida (con lo que se subsume al cálculo proposicional)
- Una expresión de la forma $A(v1, \ldots, vn)$ con $n \geq 1$ siendo cualquier v_i una variable o una constante que representa a un objeto (no a una fórmula) es una fórmula válida.
- Si A y B son fórmulas predicativas válidas
  1. $A \& B$ es una fórmula válida
  2. $A \vee B$ es una fórmula válida
  3. $\neg A$ es una fórmula válida
  4. $A \supset B$ es una fórmula válida
  5. $\forall x(A)$ es una fórmula válida
  6. $\exists x(A)$ es una fórmula válida
- No existen más fórmulas predicativas válidas que las ya definidas.

Cálculo de predicados (CPre) (III)

En fórmulas del tipo 5 ó 6 se dice que la variable x está bajo el alcance del correspondiente cuantificador ($\forall$ ó $\exists$), o que está **ligada** por él. Si una variable no está bajo el alcance de ningún cuantificador se dice que está **libre**.

Los axiomas del cálculo de predicados son los mismos (1-10) que los del cálculo proposicional, excepto en que los símbolos A, B y C pueden ser ahora también fórmulas predicativas. Además, se añaden los axiomas:

11 $\forall x A(x) \supset A(t)$

12 $A(t) \supset \exists x A(x)$

siendo t una constante o una variable libre.
Las reglas de deducción son la regla ya expresada para el cálculo proposicional, más las dos siguientes:

$$\frac{C \supset A(x)}{C \supset \forall x A(x)} \qquad \text{y} \qquad \frac{A(x) \supset C}{\exists x A(x) \supset C}$$

CPre (IV): Forma clausal

Toda fórmula válida del cálculo de predicados puede ser escrita en la llamada **forma conjuntiva normal (FCN)** que es una conjunción de disyunciones de la forma:

$$(\neg A_{11} \vee A_{12} \vee \cdots \vee \neg A_{1n_1}) \& (\neg A_{21} \vee A_{22} \vee \cdots \vee A_{2n_2}) \& .. \& (A_{m1} \vee A_{m2} \vee \neg \cdots \vee \neg A_{mn_m})$$

Se denotará en la llamada **forma clausal**: colección de cláusulas de la forma

$$A_{12} \leftarrow A_{11}, \ldots, A_{1n_1}$$
$$\ldots$$
$$A_{m1}, A_{m2} \leftarrow A_{m3}, \ldots, A_{mn_m}$$

donde los $A_{i,j}$ son fórmulas predicativas atómicas (es decir, sin conectivas lógicas) posiblemente conteniendo variables; al lado derecho de cada cláusula se sitúan las fórmulas que figuraban negadas en la FCN, y al lado izquierdo las otras.
A una cláusula de este tipo que contenga como máximo una fórmula no negada se la llama **claúsula de Horn**.

CPre (V): Paso a la forma clausal

Toda fórmula sintácticamente válida puede escribirse en forma clausual mediante una transformación por pasos. Sea el ejemplo: *Todo ser que puede leer es letrado y ningún delfín es letrado y algún delfín es inteligente y no existe algún ser que sea inteligente y que no pueda leer*.
Expresado en fórmula sería:
$\forall x(R(x) \supset L(x))\&\forall x(D(x) \supset \neg L(x))\&\exists x(D(x)\&I(x))\&\neg\exists x(I(x)\&\neg R(x))$.

- Eliminar los símbolos de implicación reemplazando $A \supset B$ por $\neg A \vee B$
  $\forall x(\neg R(x) \vee L(x))\&\forall x(\neg D(x) \vee \neg L(x))\&\exists x(D(x)\&I(x))\&\neg\exists x(I(x)\&\neg R(x))$
- Reducir el alcance de los símbolos de negación reemplazando
  - $\neg(A\&B)$ por $\neg A \vee \neg B$
  - $\neg(A \vee B)$ por $\neg A\&\neg B$
  - $\neg\forall x(A)$ por $\exists x(\neg A)$
  - $\neg\exists x(A)$ por $\forall x(\neg A)$

  $\forall x(\neg R(x) \vee L(x))\&\forall x(\neg D(x) \vee \neg L(x))\&\exists x(D(x)\&I(x))\&\forall x(\neg I(x) \vee \neg\neg R(x))$

CPre (VI): Paso a la forma clausal

- Eliminar las dobles negaciones reemplazando $\neg\neg A$ por A
  $\forall x(\neg R(x) \vee L(x)) \& \forall x(\neg D(x) \vee \neg L(x)) \& \exists x(D(x) \& I(x)) \& \forall x(\neg I(x) \vee R(x))$
- Estandarizar las variables, dando nombre único a cada variable que aparezca bajo el alcance de un cuantificador
  $\forall x(\neg R(x) \vee L(x)) \& \forall y(\neg D(y) \vee \neg L(y)) \& \exists z(D(z) \& I(z)) \& \forall u(\neg I(u) \vee R(u))$
- Sustituir las variables bajo el alcance de un cuantificador existencial por constantes o funciones de Skolem
  $\forall x(\neg R(x) \vee L(x)) \& \forall y(\neg D(y) \vee \neg L(y)) \& (D(fSk) \& I(fSk)) \& \forall u(\neg I(u) \vee R(u))$
- Convertir a forma prenexa llevando al inicio los cuantificadores universales
  $\forall x \forall y \forall u((\neg R(x) \vee L(x)) \& (\neg D(y) \vee \neg L(y)) \& (D(fSk) \& I(fSk)) \& (\neg I(u) \vee R(u)))$
- Eliminar los cuantificadores universales
  $(\neg R(x) \vee L(x)) \& (\neg D(y) \vee \neg L(y)) \& (D(fSk) \& I(fSk)) \& (\neg I(u) \vee R(u))$
- Convertir a forma conjuntiva normal
  $(\neg R(x) \vee L(x)) \& (\neg D(y) \vee \neg L(y)) \& D(fSk) \& I(fSk) \& (\neg I(u) \vee R(u))$

CPre (VII): Paso a la forma clausal

- Eliminar las conectivas de conjunción
  $\neg R(x) \vee L(x)$
  $\neg D(y) \vee \neg L(y)$
  $D(fSk)$
  $I(fSk)$
  $\neg I(u) \vee R(u)$
- Escribir cada cláusula en forma clausal
  $L(x) \leftarrow R(x)$
  $\leftarrow D(y), L(y)$
  $D(fSk) \leftarrow$
  $I(fSk) \leftarrow$
  $R(u) \leftarrow I(u)$

CPre (VIII): Algoritmo de unificación

Se dice que una fórmula A unifica con otra fórmula B mediante la sustitución S si se cumple una de estas condiciones:

- A es el término t y B es el mismo término t, con $S \equiv \{\}$
- A es un término t y B es una variable v no presente en t, con $S \equiv \{v = t\}$
- A es una variable v no presente en t y B es un término t, con $S \equiv \{v = t\}$
- A es la variable $v1$ y B es la variable $v2$, con $S \equiv \{v1 = v2\}$
- A es un predicado de la forma $f(a_1, \ldots, a_n)$,
  B es un predicado de la forma $f(b_1, \ldots, b_n)$
  y cada término a_i unifica con el correspondiente término b_i bajo la sustitución S_i, con $S = Comp_{i=1}^{n}\{S_i\}$ siendo $Comp$ la composición de sustituciones.

CPre (IX): Resolución con cláusulas Horn

Se llama **resolución** al proceso de generación de una cláusula de Horn a partir de otras dos en la forma:

$$\left.\begin{array}{l} A_1 \leftarrow B_1, \ldots, B_n \\ A_2 \leftarrow C_1, \ldots, C_m \end{array}\right\} \text{generan } As_1 \leftarrow Bs_1, \ldots, Bs_{i-1}, Bs_{i+1}, \ldots, Bs_n, Cs_1, \ldots, Cs_m$$

siempre que A_2 unifica con B_i bajo la sustitución S, siendo Fs el resultado de aplicar S a F.

Es posible probar que las tres reglas de inferencia planteadas formalmente antes son equivalentes a la resolución, en el caso de que el segundo antecedentes sea una cláusula Horn.

CPre (X): Refutación

Dada una fórmula A, y un conjunto de fórmulas $\{B_1, \ldots, B_n\}$ deseamos establecer si $B_1, \ldots, B_n \vdash A$, es decir, si A es consecuencia de (se puede deducir de) los axiomas y del conjunto dado.
Este problema es en general indecidible, es decir, no existe ningún algoritmo para saber si tal deducción existe, ni para encontrarla, aun cuando exista. No obstante, sí hay tal algoritmo para el caso particular en el que todas las B_i son clásulas Horn. Se llama **prueba por refutación**, y consiste en añadir al conjunto la negación de A, y probar que el conjunto resultante es inconsistente.
Sea $C = \{\neg A, B_1, \ldots, B_n\}$
Mientras ((sea posible resolver dos cláusulas de C) y
(no se haya generado la cláusula vacía))
Seleccionar un par de cláusulas resolubles de C
Resolverlas y añadir la resolvente a C
fin mientras
El problema es qué cláusulas seleccionar en cada paso, si varios pares son resolubles.

CPre (XI): Estrategias de resolución

Si dos cláusulas *A* y *B* resuelven para generar *C* se dice que son los padres de *C*. Una resolvente de primer nivel es aquella cuyos padres están en el conjunto base. Una resolvente de nivel i-ésimo es aquella cuyo padre de nivel más profundo está en el nivel i-1.

Se dice que una estrategia es **completa** si, en caso de existir una contradicción, se garantiza que será encontrada.

Estrategia **a lo ancho**: consiste en seleccionar todas las resolventes de primer nivel, luego todas las de segundo, etc. Es completa, pero muy ineficiente.

Estrategia del **conjunto soporte**: consiste en obtener las resolventes uno de cuyos padres sea la cláusula negación del objetivo, o alguna de las resolventes descendientes de ella (que forman el conjunto soporte). Si se explora a lo ancho todo el conjunto soporte esta estrategia es completa.

CPre (XII): Estrategias de resolución

Estrategia de **preferencia de unidades**: es una modificación de la del conjunto soporte en la que que en lugar de explorar éste a lo ancho se intenta seleccionar como uno de los padres a una cláusula con un sólo literal (llamada unidad). No es completa.

Estrategia de **entrada lineal**: es aquella en la que cada resolvente tiene al menos uno de sus padres en el conjunto inicial. Es eficiente, pero no completa. Es la usada por el lenguaje de programación PROLOG.

Estrategia de **filtrado de antepasados**: es aquella en la que cada resolvente debe tener un padre que o bien esté en el conjunto base, o bien sea un antepasado del otro padre. Es similar a la de entrada lineal, pero ésta sí es completa.

CPre (XIII): Ej. de prueba por refutación

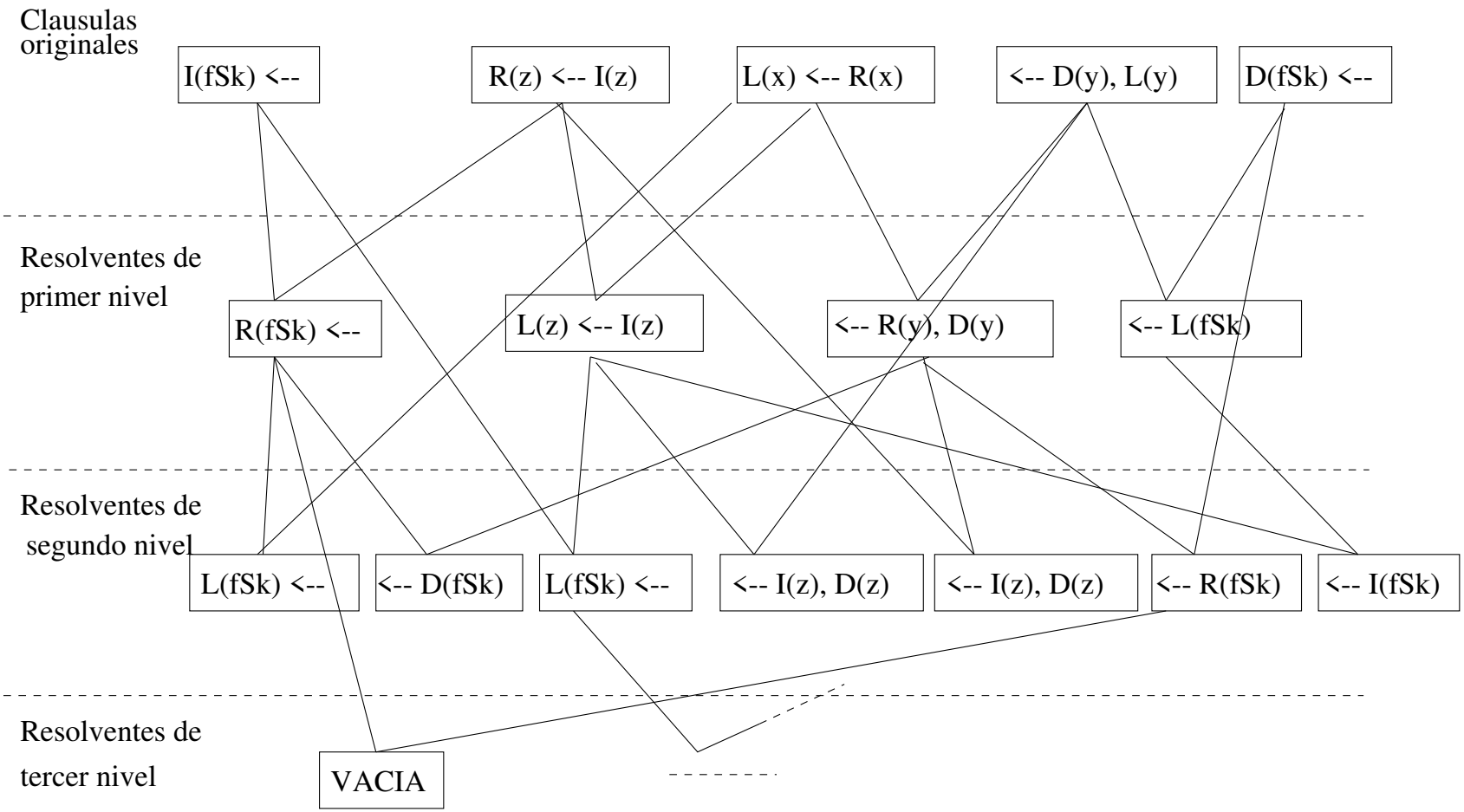
A partir de las premisas

- *"Todo ser que puede leer es letrado"*
- *"Ningún delfín es letrado"*
- *"Algún delfin es inteligente"*

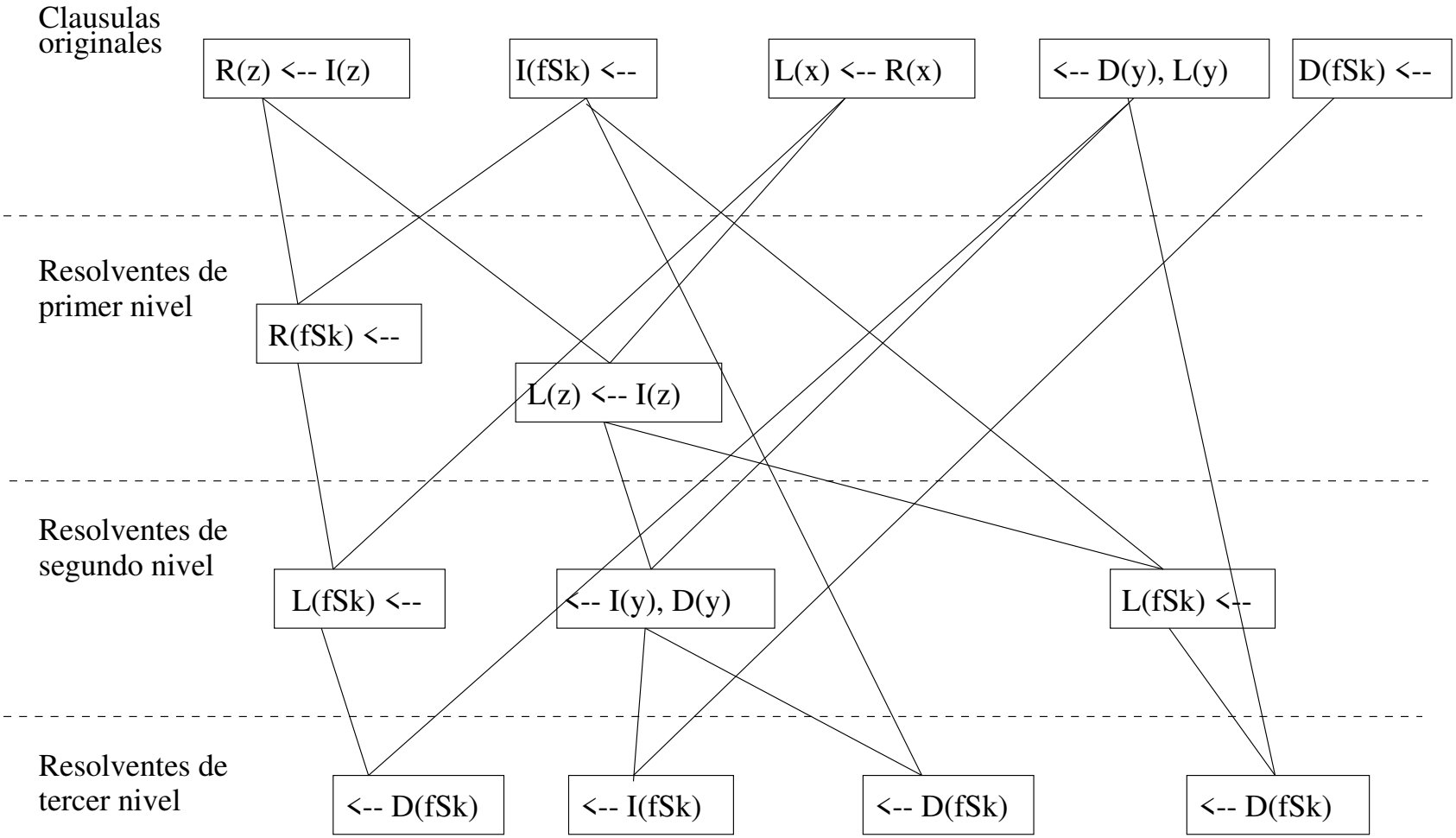
probar que *"Existe algún ser que no puede leer y es inteligente"*
En forma clausual las premisas y la negación de la conclusión quedarían

Forma clausal	Forma conjuntiva normal
$L(x) \leftarrow R(x)$	$\neg R(x) \vee L(x)$
$\leftarrow D(y), L(y)$	$\neg D(y) \vee L(y)$
$D(fSk) \leftarrow$	$D(fSk)$
$I(fSk) \leftarrow$	$I(fSk)$
$R(z) \leftarrow I(z)$	$\neg I(z) \vee R(z)$

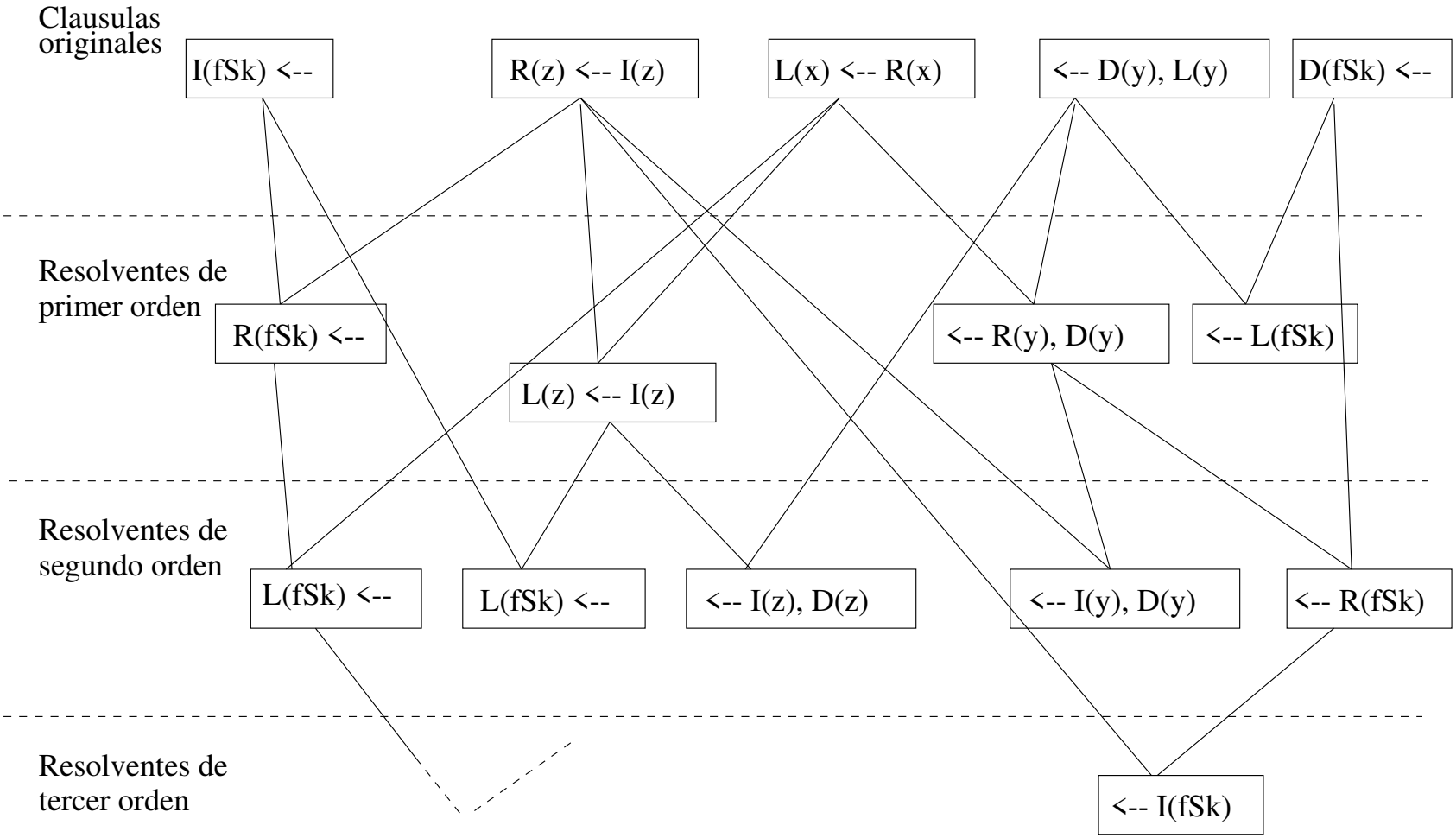
CPre (XIV): Estrategia a lo ancho



CPre (XV): Estrategia del conjunto soporte



CPre (XVI): Estrategia de entrada lineal



Sistemas de hechos y reglas (I)

La dificultad y la poca flexibilidad del razonamiento por refutación exigen buscar otros métodos que, aun sin ser completos, sean más simples y permitan cadenas de razonamiento más complejas. El intento más exitoso son los **sistemas de hechos y reglas**.
Constan de:

- Una base de hechos, proposiciones o predicados atómicos que se saben ciertos.
- Un conjunto de reglas de producción que generan (producen) más hechos ciertos a partir de la base de hechos, y de los propios hechos producidos antes.
- Un intérprete o motor de inferencia que esencialmente decide qué regla debe aplicarse en cada momento.

Sistemas de hechos y reglas (II)

Los hechos se representan en la llamada **memoria de trabajo** (WM), que puede verse como una base de datos, pero efímera.
En estos sistemas una regla tiene la forma $P \Longrightarrow Q$, que suele leerse como 'P implica Q' o bien 'Si P, entonces Q', siendo P una o un conjunto de condiciones que deben cumplirse todas, y Q bien una aserción (una proposición que se declara como cierta) o una acción (algo que cambia la memoria de trabajo o incluso el mundo real).
Hay en realidad dos posibles intepretaciones de $P \Longrightarrow Q$:

- Si P se cumple (o es cierta) entonces añadiremos el hecho de que Q se cumple (o es cierta).
  $(AfirmarP) \Longrightarrow (AfirmarQ)$
- Para que Q se verifique, debe verificarse P.
  $(ObjetivoQ) \Longrightarrow (ObjetivoP)$

Sistemas de hechos y reglas (III)

A la aplicación de $(AfirmarP) \Longrightarrow (AfirmarQ)$ se la llama **razonamiento dirigido por los datos**, **razonamiento hacia adelante** o **encadenamiento directo**.

A la aplicación de $(ObjetivoQ) \Longrightarrow (ObjetivoP)$ se la llama **razonamiento dirigido por los objetivos**, **razonamiento hacia atrás** o **encadenamiento inverso**.

Es posible usar en un mismo sistema ambos tipos de razonamiento. Incluso en lenguajes como Prolog que son por naturaleza de razonamiento hacia atrás se pueden escribir programas para implementar el razonamiento hacia adelante.

Sistemas de producción (I)

Los **sistemas de producción** son un caso particular de sist. de hechos y reglas que enfatizan el razonamiento hacia adelante.

En ellos las reglas, que se conocen como **producciones** son la forma prioritaria de representación del conocimiento.

Estos sistemas se materializaron en la forma de **sistemas expertos**.
El nombre alude a que condifican conocimiento de un área del saber concreta, y generalmente muy específica. De hecho, algunos de los ejemplos más famosos fueron

- MYCIN: para recomendación de terapia con antibióticos.
- Prospector: para recomendar lugares de prospección geológica en la búsqueda de minerales.
- R1/XCON: para la configuración de sistemas informáticos complejos (los VAX de DEC).

Sistemas de producción (II)

Idea básica de los sist. de producción:
Memoria de trabajo (WM): es como una base de datos, pero volátil.
Regla de producción:

IF
condiciones
THEN
acciones

donde 'condiciones' es un conjunto de tests sobre el estado actual de la WM, y 'acciones' una o más alteraciones de la WM.

Ciclo de operación básico:
Hacer

1. Reconocer: generar el **conjunto conflicto**, conjunto de reglas cuyas condiciones son satisfechas por el estado actual de la WM.
2. Resolver: determinar cuál de las reglas del conjunto conflicto se aplican (si hay alguna aplicable, y son más de una).
3. Actuar: efectuar los cambios en la WM determinados por las acciones de la regla escogida.

Mientras (conjunto conflicto no vacío)

Sistemas de producción (III)

La memoria de trabajo (WM) contiene elementos en ella (WME). Cada WME es de la forma

$(tipo\ \ atributo_1\ \ valor_1\ \ atributo_2\ \ valor_2\ \ \ldots\ \ atributo_n\ \ valor_n)$

Por ejemplo,
$(persona\ \ edad\ \ 27\ \ ciudad\ \ Valencia)$
$(estudiante\ \ nombre\ \ Jose_Perez\ \ asignatura\ \ Sistemas_inteligentes\ \ matricula\ \ 2)$

que se interpretan como
$\exists x(tipo(x)\&atributo_1(x, valor_1)\&atributo_2(x, valor_2)\&\ldots\&atributo_n(x, valor_n))$

El individuo x no es explícitamente mencionado en la regla; el orden de los atributos es irrelevante.

Sistemas de producción (IV)

El lado izquierdo de cada regla consiste en un conjunto de condiciones afirmadas o negadas, que se testean de modo conjunto (es decir: deben verificarse todas las afirmadas y no verificarse todas las negadas).
Cada condición es de la forma
$(tipo\ \ atributo_1\ \ especficacion_1\ \ \ldots\ \ atributo_n\ \ especificacion_n)$
donde *especificacion* es de la forma

- Un átomo (una constante literal o numérica)
- Una expresión evaluable encerrada entre []
- Una variable
- Un test encerrado entre {}
- La combinación con $\&, \vee, \neg$ de especificaciones

Sistemas de producción (V)

Ejemplos:
$(persona \;\; edad \;\; [n+4] \;\; ocupacion \;\; x)$
donde las variables n y x deberán haberse instanciado.

$\neg(persona \;\; edad \;\; \{>6 \& <23\})$

Una regla puede aplicarse **si**:
- Para cada condición afirmada, hay un elemento en la WM cuyos valores de los atributos satisfagan la condición
y además
- Para cada condición negada, **no** hay en la WM un elemento cuyos valores de los atributos la satisfagan.

Sistemas de producción (VI)

Las acciones en el lado derecho de la regla de un sistema de producción se ejecutan secuencialmente. Pueden ser de tres tipos:

- **ADD** *elemento* donde *elemento* es un WME de la forma (*type attribute value*) que se añade a la memoria de trabajo.
- **REMOVE** *indice* donde *indice* es el número de WME de la regla que se está aplicando, y lo elimina de la memoria de trabajo.
- **MODIFY** *indice* (*atributo valor*) que cambia al valor expresado el atributo expresado de un WME: aquel que tiene ese índice en la regla que se está aplicando.

Ejemplos:

IF
(Estudiante nombre x)
THEN
ADD (Persona nombre x)

Sistemas de producción (VII)

Ejemplo 1:
Colocar tres bloques en tres lugares numerados por orden de tamaño (mayor en el lugar 1, mediano en el 2, menor en el 3).

WM inicial:

(*contador indice* 1)
(*bloque nombre A tamaño* 10 *lugar mesa*)
(*bloque nombre B tamaño* 30 *lugar mesa*)
(*bloque nombre C tamaño* 20 *lugar mesa*)

Reglas de producción:

IF
(*bloque lugar mesa nombre n tamaño s*)
¬(*bloque lugar mesa tamaño* {$>$ *s*})
¬(*bloque lugar mano*)
THEN
MODIFY 1 (*lugar mano*)

IF
(*bloque lugar mano*)
(*contador indice i*)
THEN
MODIFY 1 (*lugar i*)
MODIFY 2 (*indice* [$i+1$])

(Hacer traza de la WM)

Sistemas de producción (VIII)

Ejemplo 2:

¿Cuántos días tiene un año?. La WM inicial será: $(averiguar_dias\ \ año\ \ n)$

R1 **IF**
$(averiguar_dias\ \ año\ \ n)$
THEN
REMOVE 1
ADD $(año\ \ m4\ \ [n\ \ \text{mód}\ 4]\ \ m100\ \ [n\ \ \text{mód}\ 100]\ \ m400\ \ [n\ \ \text{mód}\ 400])$

R2 **IF**
$(año\ \ m400\ \ 0)$
THEN
REMOVE 1
ADD $(tiene_dias\ \ 366)$

R3 **IF**
$(año\ \ m100\ \ 0\ \ m400\ \ \{\neq 0\})$
THEN
REMOVE 1
ADD $(tiene_dias\ \ 365)$

R4 **IF**
$(año\ \ m4\ \ 0\ \ m100\ \ \{\neq 0\})$
THEN
REMOVE 1
ADD $(tiene_dias\ \ 366)$

R5 **IF**
$(año\ \ m4\ \ \{\neq 0\})$
THEN
REMOVE 1
ADD $(tiene_dias\ \ 365)$

Sistemas de producción (IX)

Resolución de conflictos
Dadas varias reglas que es posible aplicar, ¿cuál debemos escoger?

Los criterios para hacerlo son:

- Orden de presentación (como el programa fue escrito).
- Especificidad
  - Regla con mayor número de condiciones en su lado izquierdo (más específica).
- Novedad
  - Regla que usa los WME más recientemente añadidos a la memoria.
  - Regla usada hace más ciclos.
- Variedad
  - No usar la misma regla para el mismo valor de las variables (la misma instancia de la regla).
  - Usar cada par regla/WME una sóla vez.

Sistemas de producción (X)

El uso de estos criterios depende de cada sistema concreto; dos implementaciones famosas de sistemas de hechos y reglas fueron OPS5 y SOAR.
En OPS5 los criterios (por orden de aplicación) son:

1. Descartar instancias de reglas que ya han sido usadas.
2. Ordenar las instancias restantes en términos de qué WME fue usado más recientemente.
3. Ordenar las reglas por número de condiciones (primero las de más condiciones).
4. Si aún queda más de una regla aplicable, seleccionar al azar.

Sistemas de producción (XI)

El problema de los sistemas de producción cuando el número de reglas y hechos en la WM aumenta mucho es la lentitud de comprobación del lado izquierdo (varias condiciones) de cada regla respecto a toda la WM. Sin embargo, sólo unos pocos WME cambian en cada ciclo, y además muchas reglas comparten condiciones. Esto es lo que aprovecha el algoritmo RETE.

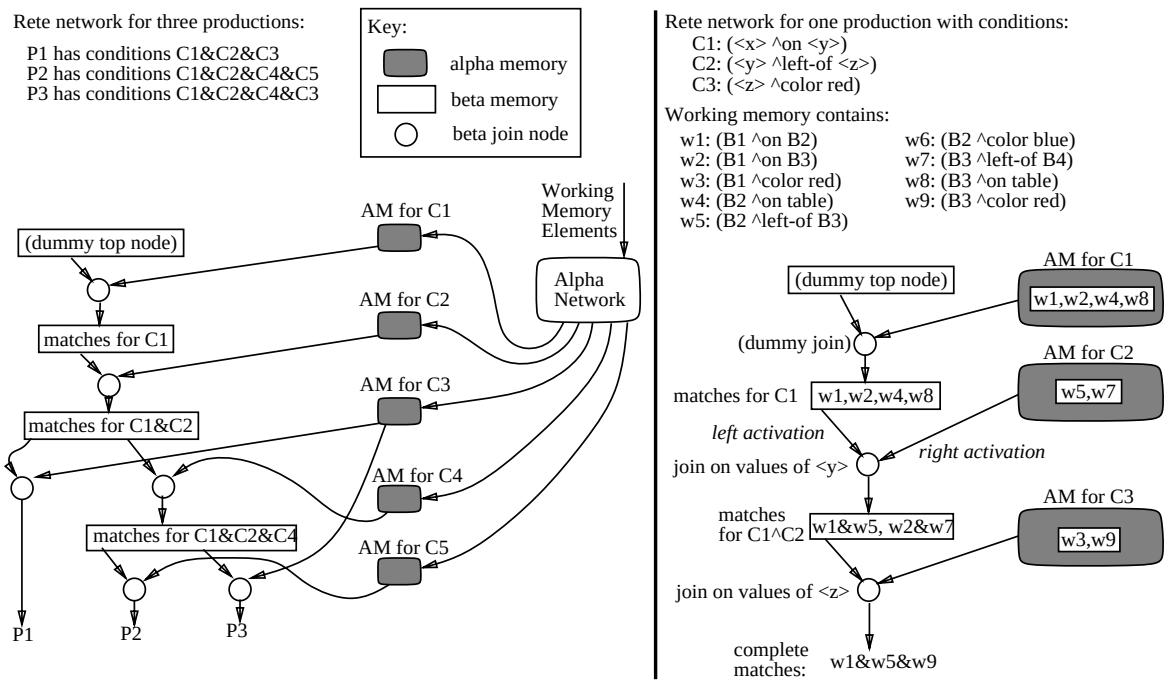
La idea de RETE es construir dos redes:

- La red α, formada por nodos (las $\alpha - memories$ o AM), uno por cada condición, independientemente de en qué regla esté. Cada uno de estos nodos contiene los hechos de la WM que satisfacen esa condición.
- La red β está formada por nodos llamados $\beta - memories$ o BM. Estos nodos se organizan en niveles y son conjunciones (o sea, & lógico) de un nodo AM y un BM de nivel anterior. Así, los BM de nivel 1 representan el encaje con una condición; los de nivel dos, con dos condiciones, etc.. A aquellos BMs que representan el encaje con todas las condiciones de una regla se les llama nodos de regla.

Sistemas de producción (XII)

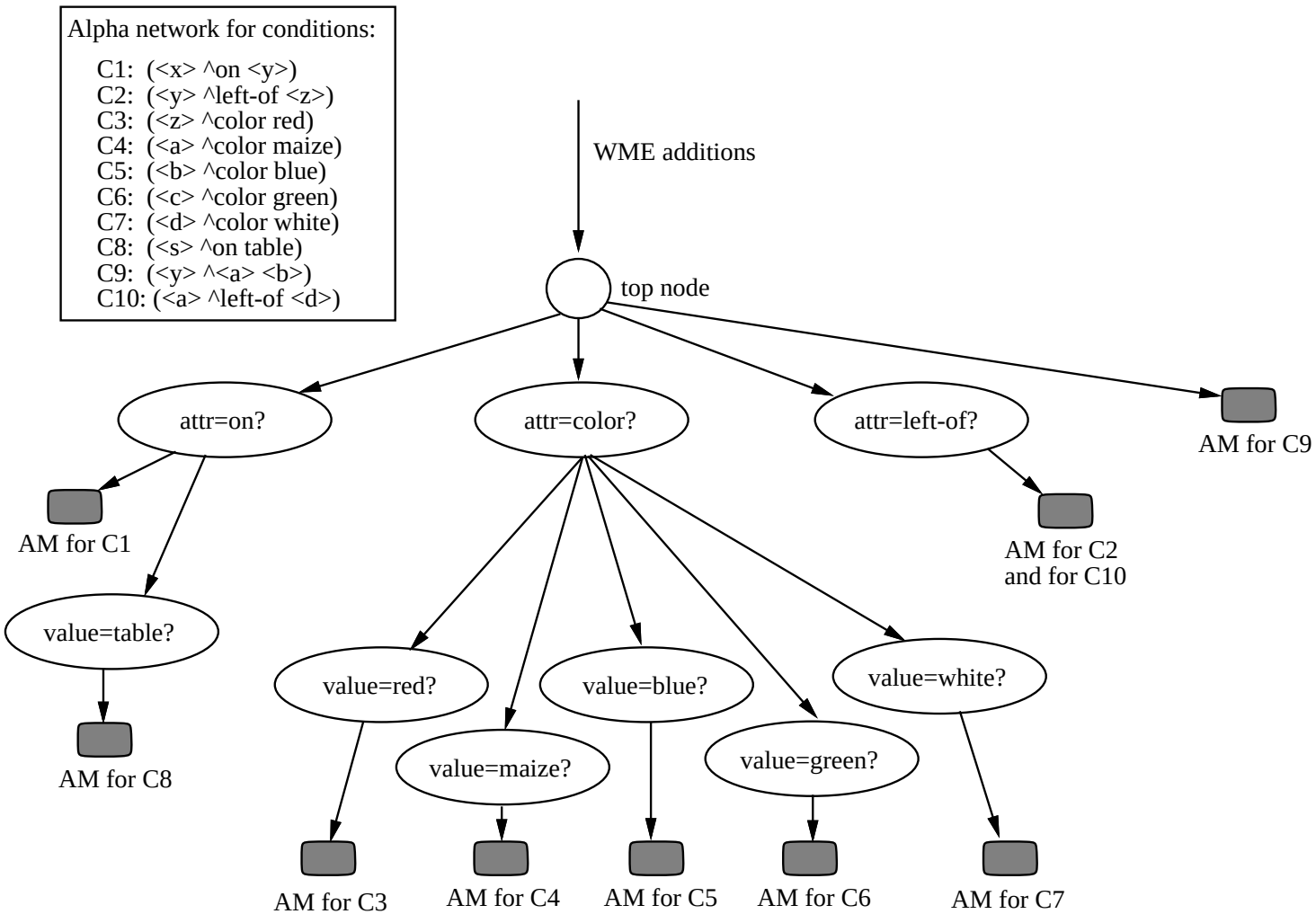
Ejemplo: Representación del mundo de bloques con hechos w1-w9 y regla P1:

```
w1: (B1 ^on B2)
w2: (B1 ^on B3)
w3: (B1 ^color red)
w4: (B2 ^on table)
w5: (B2 ^left-of B3)
w6: (B1 ^color blue)
w7: (B3 ^left-of B4)
w8: (B3 ^on table)
w9: (B3 ^color red)
P1: (find-stack-of-two-blocks-to-the-left-of-a-red-block)
      (<x> ^on <y>)
      (<y> ^left-of <z>)
      (<z> ^color red)
    –>
      ...any right-hand-side
    ) ... habrá otras reglas P2,P3...
```



Sistemas de producción (XIII)

Una posible implementación de la red α es en forma de árbol:



Sistemas de producción (XIV)

Cada vez que la WM se altera (por adición, borrado o modificación de un hecho), el algoritmo RETE propaga esta modificación, encontrando aquellas reglas que han pasado a poder aplicarse y aquellas que han dejado de poder hacerlo. Los aspectos importantes de este algoritmo que lo hacen rápido son:

- Después de cada cambio el nuevo estado se guarda en las AM y BM. Tras el siguiente cambio, muchos (incluso la mayoría) de estos resultados permanecen invariables y la mayoría de las reglas continúan en el mismo estado y no tienen que ser recomprobadas. Esto hace a RETE especialmente eficaz para sistemas donde sólo unos pocos hechos cambien tras aplicar una regla.
- Los nodos se comparten entre reglas. De este modo, la instanciación y el cambio de una condición sólo necesita hacerse una vez, independientemente de a cuántas reglas pueda afectar.
- Si no se permite la adición y/o eliminación de reglas (sólo de hechos), las AM y BM pueden precalcularse.